

UNIVERSIDAD EAFIT

Maestría en Ciencia de Datos y Analítica

SI7006 – Almacenamiento y Procesamiento de Grandes Datos

2026-1

Trabajo 3

Diseño e implementación de una solución analítica básica en una arquitectura Batch

ShiftMetrics Lakehouse

Análisis predictivo de defectos de software
sobre el paradigma Shift-Left Testing

Integrantes: Samuel Andrés Ariza Gómez, Fredy Cadavid Franco

Plataforma: Databricks Free Edition (Serverless)

Storage: Unity Catalog Volumes + Delta Lake

Procesamiento: PySpark 4.1 + SparkSQL

Machine Learning: scikit-learn + pyspark.ml.connect

Visualización: matplotlib + seaborn

Fecha de entrega: 10 de mayo de 2026

Índice

1. Introducción	3
1.1. Objetivos	3
1.2. Resumen ejecutivo de resultados	3
2. Contexto de negocio y problema	3
3. Arquitectura técnica	4
3.1. Patrón arquitectónico	4
3.2. Diagrama de la arquitectura	4
3.3. Stack tecnológico	5
3.4. Estructura física en Unity Catalog	5
4. Descripción de los datos	6
4.1. Datasets seleccionados	6
4.2. Esquema canónico de PROMISE	6
4.3. Restricciones éticas y de uso	7
5. Ingesta y capa Bronze	7
5.1. Pipeline de ingesta	7
5.2. Columnas de auditoría y trazabilidad	7
5.3. Optimización física	8
5.4. Validación: cero pérdida de datos	8
5.5. Resultado final de la capa Bronze	8
6. Preparación de datos: capa Silver	8
6.1. Objetivos y alcance	8
6.2. Transformaciones aplicadas a PROMISE	9
6.3. Transformaciones aplicadas a Red Hat	9
6.4. Constraints físicas en Delta Lake	10
6.5. La dimensión puente <code>dim_project</code>	10
6.6. Resultado final de la capa Silver	10
7. Análisis Exploratorio de Datos	11
7.1. Estrategia dual: SQL y PySpark	11
7.2. KPIs materializados como tablas Gold	11
7.3. Hallazgos del análisis	12
7.4. Visualizaciones generadas	12
8. Modelo de Machine Learning	13
8.1. Definición formal del problema	13
8.2. Arquitectura del pipeline	13
8.3. Resultados de los candidatos	13
8.4. Matriz de confusión e interpretación del trade-off	13
8.5. Importancia de features	14
8.6. Persistencia del artefacto	14
9. Capa de aplicación y visualización	14
9.1. El notebook de aplicación	15
9.2. Priorización para Shift-Left Testing	15
9.3. Inventario final del lakehouse	16

10.Decisiones de arquitectura	16
11.Reproducibilidad	18
11.1. Estructura del repositorio	18
11.2. Pasos para reproducir el lakehouse desde cero	19
11.3. Idempotencia	19
12.Limitaciones y trabajo futuro	19
13.Conclusiones	20

1. Introducción

El presente documento describe el diseño y la implementación completa de una solución analítica de Big Data construida sobre una arquitectura batch. El trabajo aplica el ciclo de vida clásico de fuentes, ingesta, almacenamiento, procesamiento, modelado y aplicación, utilizando los datasets seleccionados del Proyecto Integrador *ShiftMetrics Analytics* y consolidando la totalidad de la infraestructura sobre Databricks Free Edition.

La solución se materializa como un *lakehouse* con patrón **Medallion** canónico de Databricks, en el cual los datos transitan desde una zona **Bronze** de datos crudos inmutables, pasando por una zona **Silver** de datos limpios y conformados con *Data Quality* auditado, hasta una zona **Gold** donde residen los KPIs analíticos, las salidas del modelo de Machine Learning y las recomendaciones accionables que consume la capa de aplicación.

1.1. Objetivos

El objetivo general consiste en construir un ecosistema completo de almacenamiento y procesamiento de datos a escala bajo un patrón arquitectónico batch, demostrando dominio práctico de las tecnologías canónicas del ecosistema Spark y Delta Lake en su forma moderna sobre Unity Catalog.

Como objetivos específicos, el trabajo se propone implementar un datalake con zonas claramente diferenciadas y catalogadas; realizar la preparación, limpieza y enriquecimiento de los datos mediante DataFrames de PySpark; ejecutar un análisis exploratorio sobre los datos curados utilizando SparkSQL y materializando KPIs como tablas Delta de consumo; entrenar y evaluar un modelo de aprendizaje automático binario para la predicción de defectos a nivel de módulo de software; y, finalmente, construir una capa de aplicación que conecte directamente con la zona refinada y entregue visualizaciones y recomendaciones traducibles a decisiones de negocio.

1.2. Resumen ejecutivo de resultados

Al cierre del proyecto, el lakehouse procesa 520,415 registros distribuidos en dos dominios complementarios del ciclo de vida de desarrollo: 15,775 módulos de código provenientes del dataset PROMISE y 504,640 issues de Jira provenientes del dataset público de Red Hat. Estos datos quedan materializados en 19 tablas Delta organizadas en las tres capas del patrón Medallion, junto con un artefacto de modelo persistido en un Volume gestionado por Unity Catalog. El modelo campeón, un *RandomForestClassifier*, alcanza un AUC de 0.7316 sobre el conjunto de prueba y permite clasificar los once proyectos Apache analizados en niveles de prioridad para la adopción de Shift-Left Testing, identificando seis de ellos como críticos para inversión inmediata.

2. Contexto de negocio y problema

La industria global del desarrollo de software enfrenta una crisis estructural de calidad cuyo costo económico es medible y creciente. De acuerdo con el *Consortium for Information and Software Quality*, el software de baja calidad le costó a Estados Unidos más de 2.41 billones de dólares en 2022, y los defectos detectados tardíamente en el ciclo de vida del desarrollo constituyen el principal contribuyente a ese costo. La literatura clásica establece que corregir un defecto en producción cuesta entre diez y cien veces más que detectarlo durante la codificación, una proporción que se

amplifica en organizaciones que operan bajo esquemas de entrega continua donde la velocidad de despliegue magnifica el impacto de cada defecto que escapa hacia ambientes productivos.

El paradigma **Shift-Left Testing** emerge como respuesta a esta problemática, proponiendo mover las actividades de verificación de calidad hacia las fases más tempranas del ciclo de desarrollo. La evidencia empírica respalda su efectividad, pero su adopción organizacional exige algo que la mayoría de los equipos de ingeniería no posee: *visibilidad analítica sobre su propio proceso*, basada en datos objetivos y modelos predictivos sobre los artefactos de software que producen.

La hipótesis analítica que vertebra este trabajo afirma que las métricas estructurales del código fuente, entre ellas el acoplamiento entre objetos, la cohesión de métodos, la profundidad de la jerarquía de herencia, la complejidad ciclomática de McCabe y el número de líneas de código, son suficientemente informativas como para construir un modelo predictivo del riesgo de defectos a nivel de módulo. Si esta hipótesis se sostiene, dicho modelo permite priorizar objetivamente los proyectos que deben recibir inversión prioritaria en prácticas de Shift-Left Testing, transformando una decisión históricamente intuitiva en una decisión guiada por evidencia.

3. Arquitectura técnica

3.1. Patrón arquitectónico

La solución implementa el patrón **Medallion** canónico de Databricks, en el cual los datos transitan por tres zonas estrictamente diferenciadas. La zona **Bronze** alberga los datos en su forma cruda, sin transformación más allá del enriquecimiento con columnas de auditoría que preservan la trazabilidad del origen y la marca temporal de ingesta. Esta zona constituye la *single source of truth* inmutable del lakehouse y nunca se modifica una vez escrita. La zona **Silver** contiene los datos limpios, deduplicados y conformados, con esquema explícitamente enforced y *constraints* físicas declaradas como invariantes matemáticas sobre las columnas. Es la primera capa en la que los datos están listos para ser consumidos por analíticos y modelos. La zona **Gold** contiene los KPIs agregados, las salidas del modelo de Machine Learning y las recomendaciones accionables; constituye la API analítica del lakehouse y es la única capa que consume la aplicación final.

El enunciado del trabajo utiliza la nomenclatura “raw/bronze”, “trusted” y “refined” para referirse a estas tres zonas. La implementación presente adopta los nombres estándar de la industria Bronze, Silver y Gold, declarando explícitamente la equivalencia con los términos del enunciado: **trusted** se mapea a Silver y **refined** se mapea a Gold. Esta decisión se sustenta en que la nomenclatura Medallion es el estándar de facto en la documentación de Databricks, Apache Spark y la comunidad profesional, lo que facilita la reproducibilidad y la legibilidad del código por terceros.

3.2. Diagrama de la arquitectura

Figure 1 presenta la visión consolidada del flujo de datos desde las fuentes hasta la aplicación final. Las fuentes externas se ingestan a un Volume gestionado por Unity Catalog que funciona como zona de aterrizaje; desde allí los datos son leídos por las pipelines PySpark que materializan las tablas Delta de Bronze, transformadas posteriormente hacia Silver mediante reglas de Data Quality, y finalmente agregadas hacia Gold mediante SparkSQL y el pipeline de Machine Learning. El artefacto del modelo campeón se persiste de vuelta al Volume, cerrando el ciclo.

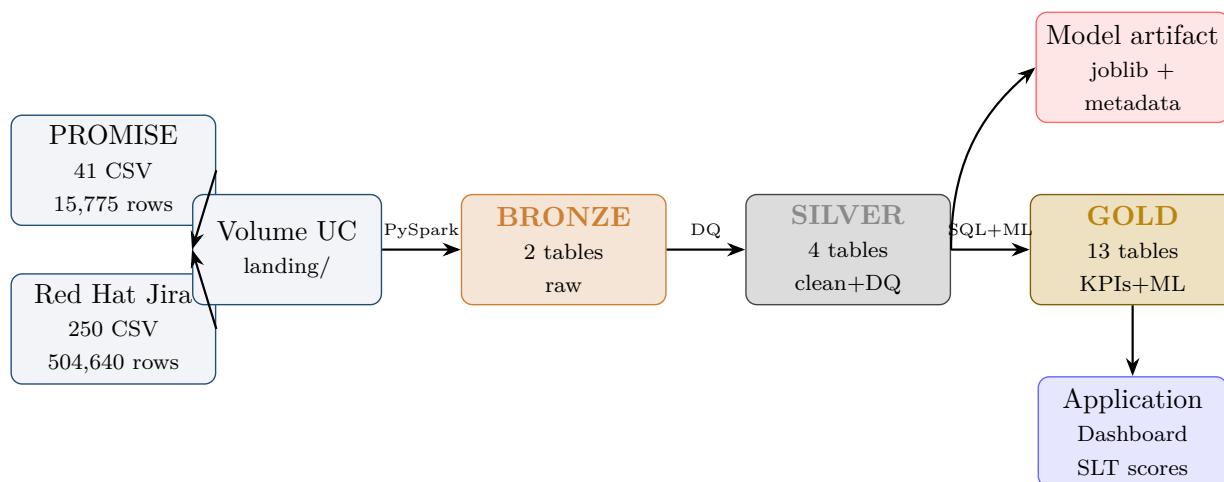


Figura 1: Arquitectura Medallion implementada. El flujo de datos avanza estrictamente de izquierda a derecha; las únicas escrituras hacia atrás corresponden al artefacto del modelo de ML, que se publica en el mismo Volume que alberga la zona de aterrizaje.

3.3. Stack tecnológico

El motor de cómputo es Databricks Free Edition operado en modalidad *Serverless* sobre Spark 4.1.0. El almacenamiento de objetos se realiza sobre un Volume gestionado por Unity Catalog, cuyo backend físico reside en S3 administrado por Databricks; sobre este almacenamiento, todas las tablas adoptan el formato *Delta Lake* con *auto-optimize* y *Z-Order* configurados como propiedades de tabla. Unity Catalog provee el catálogo de metadatos en tres niveles (`catalog.schema.table`) y enforces el contrato de tipos en toda lectura. La ingesta se orquesta vía la *Databricks CLI* versión 0.272 ejecutada localmente. El procesamiento utiliza PySpark 4.1 para las transformaciones y SparkSQL ejecutado contra el Warehouse a través de la *Statements API*. El componente de Machine Learning combina `pyspark.ml.connect` para los modelos compatibles con el runtime y `scikit-learn` para los ensambles avanzados, una decisión híbrida cuya justificación se desarrolla en section 10.

3.4. Estructura física en Unity Catalog

La estructura física del lakehouse se materializa declarativamente mediante un script de *bootstrap* idempotente que crea los tres schemas y el Volume de aterrizaje. El uso de `CREATE SCHEMA IF NOT EXISTS` y `CREATE VOLUME IF NOT EXISTS` garantiza que el script puede ejecutarse N veces sin alterar el estado del catálogo. Los *comments* declarados en cada schema son visibles desde la interfaz del Catalog Explorer y sirven como documentación inline para futuros usuarios del lakehouse.

```

CREATE SCHEMA IF NOT EXISTS workspace.shiftmetrics_bronze
  COMMENT 'Raw ingested data -- immutable source of truth';

CREATE SCHEMA IF NOT EXISTS workspace.shiftmetrics_silver
  COMMENT 'Cleansed and conformed -- schema enforced, deduplicated, typed';

CREATE SCHEMA IF NOT EXISTS workspace.shiftmetrics_gold
  COMMENT 'Business KPIs, EDA outputs and ML model artifacts';

CREATE VOLUME IF NOT EXISTS workspace.shiftmetrics_bronze.lakehouse_vol
  COMMENT 'Unified Volume: /landing, /_checkpoints, /_artifacts';
  
```

Listing 1: Bootstrap declarativo del lakehouse (idempotente).

La elección de un único Volume con sub-rutas lógicas, en lugar de tres Volumes separados por capa, responde a un principio de simplicidad operacional. Cuando los Volumes son la unidad de permisos en Unity Catalog, multiplicar Volumes multiplica también la matriz de *grants*; una sub-organización lógica con prefijos (*/landing*, */_checkpoints*, */_artifacts*) preserva la separación conceptual sin fragmentar la administración.

4. Descripción de los datos

4.1. Datasets seleccionados

El Proyecto Integrador declaró originalmente cinco candidatos a fuente de datos. Tras una evaluación técnica documentada en la primera decisión de arquitectura (DD-001 en section 10), el lakehouse se construye sobre dos de ellos. Esta selección no se realizó por preferencia arbitraria sino por afinidad técnica con la arquitectura objetivo: ambos datasets son tabulares, accesibles públicamente sin restricciones de licenciamiento, y comparten una característica fundamental del dominio que permite construir analítica complementaria sobre el ciclo de vida del desarrollo de software.

El primer dataset, **PROMISE Defect Dataset**, es el estándar de facto para benchmarking en la literatura de predicción de defectos de software. Se distribuye como repositorio GitHub público y contiene métricas estructurales de código a nivel de módulo para once proyectos del ecosistema Apache: Ant, Camel, Ivy, jEdit, log4j, Lucene, POI, Synapse, Velocity, Xalan y Xerces. Cada proyecto está representado en múltiples versiones históricas, lo que arroja un total de cuarenta y un archivos CSV y aproximadamente quince mil setecientos módulos analizables. El esquema incluye veintidós columnas: el nombre completamente cualificado de la clase, veinte métricas numéricas del catálogo Chidamber-Kemerer enriquecidas con métricas Halstead y McCabe, y una columna entera *bug* que registra el número de defectos asociados al módulo. De esta última se deriva el target binario *is_buggy* mediante la condición *bug*>0.

El segundo dataset, **Red Hat Public Jira 2001–2024**, fue publicado en Zenodo en diciembre de 2024 por investigadores de las universidades de Limerick, Galway y Carnegie Mellon. Contiene cerca de quinientos mil PBIs (*Product Backlog Items*) extraídos de 250 sistemas distintos del portafolio público de Red Hat a lo largo de veintitrés años. El esquema expone nueve columnas que describen el ciclo de vida del issue: la clave del issue, su tipo, su estado, el proyecto al que pertenece, su resolución y las marcas temporales de creación y resolución. Mientras PROMISE provee una vista *a nivel de código*, Red Hat provee una vista *a nivel de proceso*, y esta complementariedad es la razón fundamental que justifica incluir ambos en el lakehouse: permite construir una dimensión puente (*dim_project*) que unifica el análisis del SDLC desde ambos ángulos.

4.2. Esquema canónico de PROMISE

Table 1 resume las métricas más relevantes del dataset PROMISE agrupadas por familia. Las métricas Chidamber-Kemerer fueron introducidas en 1994 y constituyen el conjunto canónico para evaluar la calidad de un diseño orientado a objetos; capturan dimensiones como complejidad estructural (WMC), profundidad de herencia (DIT), acoplamiento entre clases (CBO), cohesión interna (LCOM) y la respuesta total de la clase ante una invocación externa (RFC). Las métricas Halstead aportan información sobre el tamaño y la densidad léxica del módulo, y las métricas McCabe capturan la complejidad ciclomática promedio y máxima del flujo de control. El target *bug*, al binarizarse, produce una distribución desbalanceada moderada con aproximadamente 36 % de módulos defectuosos, lo cual constituye una señal adecuada para la clasificación binaria.

Métrica	Familia	Tipo	Descripción
wmc	C&K	double	Weighted Methods per Class
dit	C&K	double	Depth of Inheritance Tree
noc	C&K	double	Number of Children
cbo	C&K	double	Coupling Between Objects
rfc	C&K	double	Response For Class
lcom	C&K	double	Lack of Cohesion of Methods
loc	Halstead	double	Lines of Code
max_cc	McCabe	double	Maximum Cyclomatic Complexity
avg_cc	McCabe	double	Average Cyclomatic Complexity
bug	Target	int	Número de bugs (binarizable)

Cuadro 1: Subset representativo de las veintidós columnas del esquema PROMISE.

4.3. Restricciones éticas y de uso

Conforme a las restricciones documentadas en el PI ShiftMetrics, los datasets se procesan exclusivamente a nivel de módulo o sistema agregado. El dataset Red Hat contiene en algunas versiones información de identificación personal de desarrolladores, por lo cual se utiliza únicamente la versión anonimizada publicada en Zenodo y, además, ninguna métrica del lakehouse se agrega a nivel individual de desarrollador. Cualquier análisis de productividad personal queda explícitamente fuera del alcance de este trabajo, decisión que se documenta también en el `README.md` del repositorio.

5. Ingesta y capa Bronze

5.1. Pipeline de ingesta

La ingesta se ejecuta como un proceso reproducible en tres etapas. La primera consiste en la descarga local de los archivos crudos: PROMISE se obtiene mediante `curl` contra el `raw` de GitHub usando un script de bash que itera explícitamente sobre la matriz de proyectos y versiones verificada previamente contra el repositorio; Red Hat se obtiene resolviendo los enlaces de descarga mediante la API REST de Zenodo. La segunda etapa es una fase de saneamiento previa a la subida al Volume: se descomprimen los archivos ZIP de Zenodo, se eliminan los archivos de `lock` de Excel (`~$*`) que aparecen como artefactos de los autores originales, y se aplica la exclusión de `RHD.csv` documentada en DD-002. La tercera etapa es la subida al Volume mediante `databricks fs cp -recursive -overwrite`, que es idempotente y permite re-ejecutar la subida sin riesgo de duplicación.

Una vez los archivos residen en el Volume, el notebook `01_ingest_bronze` los lee mediante PySpark con esquema explícito (no se utiliza `inferSchema` en ningún momento, decisión deliberada que garantiza fail-fast ante datos malformados) y los persiste como tablas Delta.

5.2. Columnas de auditoría y trazabilidad

Las dos tablas Bronze se enriquecen con tres columnas de auditoría obligatorias: `_source_file` preserva la ruta absoluta del archivo del cual proviene cada fila, `_ingestion_ts` registra la marca temporal de ingesta, y un identificador derivado del path (`_project` y `_version` en PROMISE,

`_system_bucket` en Red Hat) permite reconstruir la procedencia sin necesidad de parsear la ruta. La obtención de la ruta del archivo merece una nota técnica: el método clásico de PySpark, la función `input_file_name()`, está prohibido en Unity Catalog Strict por razones de seguridad. La implementación utiliza la API moderna `F.col("_metadata.file_path")`, que no solo es la única autorizada bajo Unity Catalog sino que es estrictamente tipada y richer que su predecesora.

5.3. Optimización física

Cada tabla Bronze se escribe particionada por su clave de mayor cardinalidad para selectividad de lectura (PROMISE por `_project`, Red Hat por `_system_bucket` construido como la primera letra del project key, lo cual mantiene la cardinalidad acotada a 26 particiones máximo). Adicionalmente se aplican las propiedades `delta.autoOptimize.optimizeWrite` y `delta.autoOptimize.autoCompact` y se ejecuta `OPTIMIZE ... ZORDER BY` sobre las columnas de filtrado frecuente. El *Z-Order* en Bronze prepara los datos para las lecturas selectivas que ejecutará Silver, evitando *full scans* en transformaciones posteriores.

5.4. Validación: cero pérdida de datos

La validación de la capa Bronze no se reduce a confiar en que “el ingest funcionó”: se ejecuta un *diagnostic* explícito que compara el conteo real de filas en los archivos CSV del Volume contra el conteo en la tabla Delta resultante, agrupando por (`project`, `version`). Este diagnostic es código auditable que se conserva en el notebook como evidencia de integridad. Table 2 presenta el resultado: cero filas perdidas en la totalidad del proceso.

Verificación	Esperado	Obtenido	Diferencia
PROMISE – filas en archivos CSV	15,775	15,775	0
PROMISE – filas en tabla Delta	15,775	15,775	0
Red Hat – filas en archivos CSV	504,640	504,640	0
Red Hat – filas en tabla Delta	504,640	504,640	0

Cuadro 2: Validación de integridad Bronze. El diagnostic compara fila a fila los CSV del Volume contra la tabla Delta resultante.

5.5. Resultado final de la capa Bronze

Table 3 resume la distribución de los datos PROMISE por proyecto Apache tras la ingesta. La tasa de defectuosidad varía significativamente entre proyectos, desde un 16.9% en Ivy hasta un 57.9% en log4j, lo cual constituye un primer indicio favorable para el modelo predictivo: ningún proyecto es trivial en uno u otro sentido, y por tanto existe señal heterogénea para que el modelo aprenda.

6. Preparación de datos: capa Silver

6.1. Objetivos y alcance

La capa Silver convierte los datos crudos de Bronze en datos confiables listos para análisis y modelado. Cuatro tablas se materializan en este schema: `code_metrics_clean` con el PROMISE

Proyecto	Módulos	Versiones	Buggy	% buggy
ant	1,692	5	350	20.7
camel	2,784	4	562	20.2
ivy	704	3	119	16.9
jedit	1,749	5	303	17.3
log4j	449	3	260	57.9
lucene	782	3	438	56.0
poi	1,378	4	707	51.3
synapse	635	3	162	25.5
velocity	639	3	367	57.4
xalan	3,320	4	1,806	54.4
xerces	1,643	4	654	39.8
Total	15,775	—	5,728	36.3

Cuadro 3: Distribución de defectos por proyecto Apache tras la ingesta Bronze. La heterogeneidad de tasas garantiza señal aprovechable por el modelo de ML.

depurado y enriquecido, `process_issues_clean` con el Red Hat depurado, `dim_project` como dimensión puente que unifica los dos dominios, y `_dq_metrics` como bitácora persistente y auditable de todos los chequeos de calidad ejecutados.

La conversión Bronze→Silver no es una limpieza mecánica sino una serie de decisiones explícitas sobre qué es un dato válido, qué se descarta y qué se transforma. Cada decisión queda registrada como un evento en `_dq_metrics` con su nivel de severidad (*critical*, *warning* o *info*), el valor esperado, el valor observado y un `run_id` único por ejecución, lo cual permite auditar cualquier corrida histórica y detectar regresiones entre ejecuciones sucesivas.

6.2. Transformaciones aplicadas a PROMISE

El pipeline para PROMISE comienza con un *pre-DQ* sobre Bronze que cuenta nulos en las columnas críticas (`name`, `loc`, `bug`) y filas con valor negativo en `bug`; en la corrida real, las cuatro verificaciones resultaron en cero, confirmando la calidad estructural de PROMISE. A continuación se aplica un filtro defensivo (`isNotNull` y `bug>=0`) que actúa como red de seguridad incluso si los chequeos pre-DQ fueron positivos. La deduplicación se realiza mediante una window function que asigna `row_number` sobre la clave natural (`name`, `_project`, `_version`) ordenando por `_ingestion_ts` descendente, y se conserva únicamente la fila más reciente; este patrón soporta correctamente el caso en el que un mismo módulo aparezca en múltiples ingestas. Finalmente se generan las columnas derivadas `is_buggy` como binarización del campo `bug` y `bug_density` como cociente `bug/loc` que captura la concentración relativa de defectos por línea de código.

6.3. Transformaciones aplicadas a Red Hat

El pipeline para Red Hat incluye una transformación adicional crítica derivada de un hallazgo de calidad: 125 issues del dataset presentaban `resolved<created`, una condición físicamente imposible que sugiere errores de captura o retroactividad manual en los Jira originales. Estas filas se filtran explícitamente y la decisión queda registrada como DD-003 y como un evento `rows_dropped_inverted_dates` en `_dq_metrics`. Junto con la deduplicación por `issue_key` y el casting de timestamps (necesario porque las fechas vienen en formato europeo `dd/MM/yyyy HH:mm`,

que SparkSQL no infiere automáticamente), se construyen las columnas derivadas que habilitan el análisis temporal: `year_created`, `month_created`, `is_resolved`, `cycle_time_days` y `age_days`.

6.4. Constraints físicas en Delta Lake

Más allá de los chequeos lógicos en el pipeline, Silver declara seis *CHECK constraints* físicas sobre las tablas Delta. Una constraint Delta es una expresión booleana que se verifica en cada escritura futura sobre la tabla; cualquier intento de insertar una fila que la viole es rechazado por el motor. Esto convierte las invariantes de negocio en invariantes físicas del almacenamiento. Table 4 las resume.

Tabla Silver	Constraint	Expresión
<code>code_metrics_clean</code>	<code>bug_non_negative</code>	<code>bug >= 0</code>
<code>code_metrics_clean</code>	<code>loc_non_negative</code>	<code>loc >= 0</code>
<code>code_metrics_clean</code>	<code>is_buggy_binary</code>	<code>is_buggy IN (0,1)</code>
<code>process_issues_clean</code>	<code>created_not_future</code>	<code>created <= current_timestamp()</code>
<code>process_issues_clean</code>	<code>cycle_time_non_negative</code>	<code>cycle_time_days IS NULL OR cycle_time_days >= 0</code>
<code>process_issues_clean</code>	<code>is_resolved_binary</code>	<code>is_resolved IN (0,1)</code>

Cuadro 4: Constraints Delta declaradas en Silver. Son invariantes físicas verificadas en cada escritura futura sobre las tablas.

6.5. La dimensión puente `dim_project`

PROMISE describe proyectos Apache y Red Hat describe sistemas Red Hat; ninguno de los dos comparte claves naturales con el otro. La dimensión `dim_project` resuelve este desencuentro materializando una clave sintética `project_id` de la forma `<source>:<key>`, donde `source` identifica el dominio (`apache` o `redhat`) y `key` es la clave natural del proyecto en ese dominio. La tabla resultante contiene 261 filas (11 proyectos Apache más 250 sistemas Red Hat) y permite construir consultas cross-domain que comparan métricas de ambos universos bajo un modelo dimensional único.

Es importante señalar que esta dimensión *no* pretende afirmar que los proyectos Apache y los sistemas Red Hat sean comparables uno-a-uno; afirma que pertenecen al mismo dominio conceptual (proyectos de software con métricas medibles) y por tanto pueden analizarse bajo un esquema unificado. La transparencia sobre este punto se mantiene mediante la columna `domain_layer`, que distingue explícitamente entre la vista `code` de Apache y la vista `process` de Red Hat.

6.6. Resultado final de la capa Silver

Table 5 presenta el resumen consolidado de retención y Data Quality tras la transformación. La retención del 100 % en PROMISE indica que el dataset es estructuralmente limpio; la retención del 98.94 % en Red Hat refleja la combinación de filas con `issue_key` nulo, fechas no parseables y fechas invertidas que el pipeline eliminó. Los diecinueve chequeos de Data Quality ejecutados produjeron cero fallas críticas y una sola advertencia, correspondiente a la condición `resolved<created` que el filtro DD-003 ya gestiona.

Dataset	Métrica	Bronze	Silver
PROMISE	Filas	15,775	15,775
PROMISE	Retención	—	100.00 %
Red Hat	Filas	504,640	499,313
Red Hat	Retención	—	98.94 %
Red Hat	Eliminadas por <code>resolved<created</code>	—	125
Red Hat	Eliminadas por otras reglas DQ	—	5,202
DQ checks ejecutados		—	19
Resultados FAIL		—	0
Resultados WARN (documentados)		—	1

Cuadro 5: Resumen de retención y Data Quality en Silver.

7. Análisis Exploratorio de Datos

7.1. Estrategia dual: SQL y PySpark

El enunciado del trabajo solicita expresamente que el EDA se realice “utilizando SQL, SparkSQL y PySpark”. Para cumplir con ambas modalidades de forma rigurosa, el EDA se separa en dos notebooks complementarios. El primero, `03_eda_with_sql.sql`, contiene exclusivamente sentencias SparkSQL ejecutadas contra el Warehouse vía la Statements API, y materializa ocho KPIs como tablas Delta independientes en el schema Gold. El segundo, `04_eda_pyspark_visualization.py`, utiliza DataFrames PySpark para el profiling estadístico distribuido y matplotlib más seaborn para la generación de ocho visualizaciones inline. La separación no es formal sino funcional: la versión SQL produce los datos analíticos persistentes que luego son consumidos por la versión Python, demostrando una integración real entre ambos lenguajes.

7.2. KPIs materializados como tablas Gold

Table 6 resume los ocho KPIs analíticos materializados en Gold. Cada uno se declara como tabla Delta *managed* con su comment descriptivo, lo cual significa que su definición de negocio queda visible en el Catalog Explorer sin necesidad de consultar el SQL fuente.

Tabla Gold	Propósito
<code>kpi_defect_distribution_by_project</code>	Tasa de defectos por proyecto Apache.
<code>kpi_metric_correlations_with_target</code>	Correlación de Pearson de cada feature numérica con <code>bug</code> e <code>is_buggy</code> .
<code>kpi_top_buggy_modules</code>	Top 25 módulos con mayor número absoluto de defectos.
<code>kpi_defect_density_quintiles</code>	Distribución quintílica de <code>bug_density</code> por proyecto.
<code>kpi_throughput_yearly</code>	Volumen, tasa de resolución y cycle time anual de Red Hat.
<code>kpi_cycle_time_by_issue_type</code>	Distribución del cycle time según tipo de issue.
<code>kpi_redhat_top_systems_by_volume</code>	Top 20 sistemas Red Hat con mayor volumen de issues.
<code>kpi_cross_domain_summary</code>	Vista unificada PROMISE–Red Hat vía <code>dim_project</code> .

Cuadro 6: KPIs materializados como tablas Delta en el schema Gold.

7.3. Hallazgos del análisis

El análisis de correlación entre cada feature numérica de PROMISE y el target `is_buggy` revela un patrón consistente con la teoría clásica de la ingeniería de software empírica. Las cinco features de mayor correlación absoluta son `rfc`, `npm`, `wmc`, `loc` y `cam`, todas ellas relacionadas con tamaño y complejidad de la clase; `cam`, que mide la cohesión interna de los métodos, presenta el único signo negativo, lo cual confirma la intuición teórica de que una mayor cohesión protege contra los defectos. Los coeficientes individuales son moderados (la mayor magnitud absoluta es 0.20), pero su consistencia y la significancia estadística que conlleva tener 15,775 observaciones justifican el uso de un modelo no lineal capaz de capturar interacciones entre las features.

Hallazgo clave: Rango ideal para clasificación binaria

La tasa de defectos por proyecto se distribuye entre 16.9 % y 57.9 %, sin proyectos triviales en ninguno de los dos extremos. Esto garantiza una señal aprovechable para el modelo de ML: ningún proyecto degenera hacia clasificación de clase única, y la heterogeneidad entre proyectos permite que el modelo aprenda patrones generalizables.

El análisis temporal sobre Red Hat revela tres patrones macroscópicos del ciclo de vida de issues entre 2010 y 2024 que se sintetizan en table 7. El volumen anual creció ocho veces, pasando de aproximadamente diez mil issues anuales en 2010 a más de ochenta mil en 2024, lo cual refleja la expansión del portafolio open-source de Red Hat. La tasa de resolución decayó de 96.3 % a 63.2 %, sugiriendo que el volumen creciente excede la capacidad de procesamiento de los equipos. El cycle time promedio aparenta haber mejorado de 207 días a 42, pero esta mejora es un artefacto estadístico de *right-censoring*: los issues de años recientes que aún no se han resuelto no contribuyen al promedio, sesgándolo hacia abajo. Este es un hallazgo importante que debe mencionarse al consumidor del KPI para evitar conclusiones equívocas.

Año	Issues	% resueltos	Avg cycle (d)	Mediana	p95
2010	10,044	96.3	207.4	13	1,247
2015	19,876	94.7	220.1	23	1,156
2020	28,472	88.9	171.6	34	814
2023	82,935	83.2	90.3	39	543
2024	81,125	63.2	42.0	20	247

Cuadro 7: Evolución temporal Red Hat. La aparente mejora del cycle time en 2024 es un artefacto de *right-censoring* sobre issues recientes aún no resueltos.

7.4. Visualizaciones generadas

El notebook PySpark genera ocho visualizaciones inline cubriendo las distribuciones, las correlaciones y las series temporales relevantes para el dominio. Cada figura está embebida en formato PNG base64 dentro del notebook HTML exportado y se conserva en el directorio `notebooks/exports/` del repositorio. La lista de figuras incluye el bar chart de tasa de defectos por proyecto, el boxplot logarítmico de densidad de bugs por proyecto, las quince clases más defectuosas en barras horizontales, el heatmap de correlación entre features y target, la evolución dual-axis de Red Hat con barras apiladas y línea de tasa de resolución, la banda de cycle time con mediana, mean y percentil 95, las barras horizontales del cycle time por tipo de issue, y el panel doble de comparación cross-domain entre Apache y Red Hat.

8. Modelo de Machine Learning

8.1. Definición formal del problema

Se plantea un problema de clasificación binaria supervisada en el cual una función $f : \mathbb{R}^{20} \rightarrow \{0, 1\}$ recibe el vector de veinte métricas estructurales del módulo y devuelve una predicción sobre el target binario `is_buggy`. La función real entrenada es de hecho $f : \mathbb{R}^{20} \rightarrow [0, 1]$, una probabilidad calibrada que se discretiza con `threshold 0.5` únicamente para reportar matrices de confusión y métricas dependientes del umbral. La métrica primaria de optimización es el AUC bajo la curva ROC, que es invariante al `threshold` y captura la capacidad discriminativa del modelo en su conjunto.

8.2. Arquitectura del pipeline

El pipeline de ML consta de cinco etapas secuenciales. La primera es la ingeniería de features, implementada íntegramente en SparkSQL: a partir del conjunto de entrenamiento se calculan la media y la desviación estándar de cada feature, y estos parámetros se aplican como transformación de z-score sobre ambos conjuntos (entrenamiento y prueba), evitando explícitamente la fuga de información del conjunto de prueba hacia los parámetros de escalado. La segunda etapa es el *split* estratificado 80/20 mediante `sampleBy`, que preserva la distribución del target en ambos conjuntos. La tercera etapa entrena tres candidatos con *cross-validation* de cinco *folds* y búsqueda en *grid* de hiperparámetros. La cuarta selecciona el modelo campeón por mayor AUC en el conjunto de prueba. La quinta persiste el campeón en el Volume y genera las cuatro tablas Gold con las métricas, predicciones, matriz de confusión e importancia de features.

8.3. Resultados de los candidatos

Table 8 presenta el desempeño de los tres candidatos sobre el conjunto de prueba. El Random-Forest obtuvo el mayor AUC y se designa como modelo campeón; el Gradient Boosting quedó marginalmente por debajo (diferencia de 0.0008 en AUC, estadísticamente indistinguible). La LogisticRegression bajo `pyspark.ml.connect`, que era el candidato representante de la API distribuida nativa de SparkMLlib, no pudo entrenarse exitosamente en el runtime de Databricks Free Edition; la explicación técnica detallada se documenta en DD-004 (ver section 10).

Modelo	AUC	F1	Acc.	Prec.	Rec.	Framework
RandomForest	0.7316	0.6869	0.7112	0.7010	0.7112	scikit-learn
GradientBoosting	0.7308	0.6856	0.7099	0.6995	0.7099	scikit-learn
LogisticRegression	—	—	—	—	—	<code>spark.ml.connect</code>

Cuadro 8: Comparación de modelos en el conjunto de prueba. El RandomForest es el campeón. LogisticRegression no pudo ejecutarse debido a las restricciones del runtime documentadas en DD-004.

8.4. Matriz de confusión e interpretación del trade-off

La matriz de confusión del modelo campeón sobre el conjunto de prueba (3,109 módulos) presenta 1,799 verdaderos negativos, 215 falsos positivos, 683 falsos negativos y 412 verdaderos positivos. Esto se traduce en una precisión sobre la clase positiva de 0.657 y un recall de 0.376, con un F1 positivo de 0.479. El modelo es por tanto *conservador*: cuando declara un módulo como defectuoso

acierta en dos de cada tres ocasiones, pero deja sin detectar aproximadamente seis de cada diez módulos que efectivamente son defectuosos.

Hallazgo clave: Implicaciones operativas del trade-off

El perfil precision/recall del modelo lo hace adecuado para tareas de priorización (“en qué proyectos invertir SLT primero”), donde un falso positivo es un costo aceptable de tiempo de revisión, pero lo hace insuficiente como única señal de *gate* en una pipeline CI/CD, donde un falso negativo significa pasar un defecto a producción. Para uso productivo en CI/CD sería necesario re-balanceo de clases (por ejemplo SMOTE o ajuste de `class_weight`) o tuning del threshold para favorecer recall.

8.5. Importancia de features

La función `feature_importances_` del RandomForest produce el ranking de table 9. La feature dominante es `loc` (líneas de código), seguida por `amc` (complejidad promedio del método), `rfc` (response for class), `cam` (cohesión entre métodos) y `cbm` (acoplamiento entre métodos). Este ranking confirma la lectura clásica de la calidad orientada a objetos: el tamaño y la complejidad ciclomática del módulo, sumados a su acoplamiento con el resto del sistema, dominan la propensión a defectos.

Rank	Importancia	Feature
1	0.141	<code>loc</code> – Lines of Code
2	0.097	<code>amc</code> – Average Method Complexity
3	0.073	<code>rfc</code> – Response For Class
4	0.071	<code>cam</code> – Cohesion Among Methods
5	0.058	<code>cbm</code> – Coupling Between Methods

Cuadro 9: Top 5 features según el modelo campeón.

8.6. Persistencia del artefacto

El modelo campeón se serializa con `joblib` y se persiste en el Volume junto con el escalador correspondiente y un archivo `metadata.json` que registra el `run_id`, las métricas de entrenamiento, los hiperparámetros seleccionados y la lista canónica de features. La estructura de directorios resultante es la siguiente:

```
/Volumes/workspace/shiftmetrics_bronze/lakehouse_vol/_artifacts/models/
  champion_20260510_HHMMSS/
    sklearn_model.joblib      (RandomForest serializado)
    scaler.joblib             (StandardScaler de inferencia)
    metadata.json             (run_id, metricas, hiperparametros)
```

Esta estructura permite a un consumidor downstream cargar el modelo y ejecutar inferencia sin necesidad de re-entrenarlo, lo cual es el patrón canónico de *model serving* en arquitecturas MLOps.

9. Capa de aplicación y visualización

9.1. El notebook de aplicación

El notebook `06_application_dashboard.py` constituye la última capa del lakehouse y la única que el consumidor final (un Engineering Manager, un Product Owner o un CTO) interactúa directamente. Su única responsabilidad es *consumir* las tablas del schema Gold y presentar la información en un formato decisonal, sin ejecutar ninguna transformación de los datos crudos. Esta separación de responsabilidades es deliberada: garantiza que cualquier cambio en la lógica de presentación no toca los datos analíticos, y que cualquier cambio en los datos analíticos se refleja automáticamente en la aplicación sin necesidad de re-codificar las visualizaciones.

El notebook se estructura en siete secciones lógicas. La primera presenta un *executive summary* con los KPIs principales en bloque de texto, diseñado para ser leído en quince segundos. La segunda contrasta la tasa histórica de defectos por proyecto con la tasa predicha por el modelo, permitiendo identificar visualmente proyectos donde el modelo discrepa del histórico. La tercera es un dashboard de tres paneles sobre el modelo campeón: la curva ROC, la matriz de confusión en heatmap y la importancia de features. La cuarta presenta el estado del proceso Red Hat con su evolución temporal. La quinta sintetiza la comparación cross-domain entre Apache y Red Hat vía `dim_project`. La sexta calcula y persiste la priorización final para SLT. La séptima cierra con el inventario completo del lakehouse.

9.2. Priorización para Shift-Left Testing

El *output* más importante del notebook de aplicación es la tabla `app_slt_prioritization`, que se persiste en Gold como API accionable para cualquier sistema downstream. Su lógica es directa pero combina las dos fuentes de información disponibles: la tasa histórica documentada por proyecto y la tasa de defectos que predice el modelo sobre los módulos del conjunto de prueba. El score combinado es el promedio aritmético simple de ambas tasas, lo cual asigna pesos equivalentes al pasado observado y al futuro predicho. Sobre este score se aplica una tabla de tiers que clasifica los proyectos en P0 (crítico, si el score supera 50), P1 (alto, entre 30 y 50), P2 (medio, entre 20 y 30) y P3 (bajo, por debajo de 20).

Table 10 presenta la priorización final. Seis de los once proyectos Apache caen en la categoría P1 y deben ser candidatos prioritarios para inversión en Shift-Left Testing. Los cinco proyectos restantes se ubican en P2 y deben recibir atención secundaria.

Proyecto	Módulos	Hist %	Pred %	Score	Priority
xalan	3,320	54.4	45.2	49.8	P1 – HIGH
poi	1,378	51.3	45.8	48.5	P1 – HIGH
lucene	782	56.0	38.2	47.1	P1 – HIGH
log4j	449	57.9	33.6	45.8	P1 – HIGH
velocity	639	57.4	32.8	45.1	P1 – HIGH
xerces	1,643	39.8	32.5	36.1	P1 – HIGH
synapse	635	25.5	31.6	28.6	P2 – MEDIUM
ant	1,692	20.7	33.1	26.9	P2 – MEDIUM
jedit	1,749	17.3	33.5	25.4	P2 – MEDIUM
ivy	704	16.9	33.1	25.0	P2 – MEDIUM
camel	2,784	20.2	27.1	23.6	P2 – MEDIUM

Cuadro 10: Priorización SLT por proyecto, persistida en `gold.app_slt_prioritization`. Seis proyectos en categoría P1.

Hallazgo clave: El modelo discrepa del histórico en proyectos pequeños

Para *jedit*, el histórico documenta solamente 17.3 % de módulos defectuosos, pero el modelo predice 33.5 %. La discrepancia indica que el modelo detecta patrones de código riesgoso que aún no se han manifestado como bugs registrados en la versión presente. Esto es exactamente el valor que aporta Shift-Left Testing: anticipar antes del incidente, no reaccionar después.

9.3. Inventario final del lakehouse

Al cierre del proyecto el lakehouse alberga diecinueve tablas Delta distribuidas en los tres schemas del patrón Medallion, más un artefacto de modelo persistido en el Volume. Table 11 consolida el inventario.

Capa	Tablas	Total
Bronze	code_metrics_raw, process_issues_raw	2
Silver	code_metrics_clean, process_issues_clean, dim_project, _dq_metrics	4
Gold (EDA)	ocho tablas kpi_*	8
Gold (ML)	ml_model_metrics, ml_predictions, ml_confusion_matrix, ml_feature_importance	4
Gold (App)	app_slt_prioritization	1
Total	—	19

Cuadro 11: Inventario final del lakehouse al cierre del proyecto.

10. Decisiones de arquitectura

A lo largo del desarrollo se tomaron cuatro decisiones de arquitectura no triviales, cada una de las cuales modificó significativamente el diseño o el alcance del lakehouse. Todas se documentan a continuación siguiendo un formato uniforme de contexto, decisión adoptada, alternativas evaluadas y consecuencias aceptadas, en línea con el patrón canónico de *Architecture Decision Records*.

DD-001 Selección de fuentes de datos

Contexto. El Proyecto Integrador ShiftMetrics declaró cinco fuentes candidatas: PROMISE, Apache Jira (volcado MongoDB de aproximadamente 2 GB), Red Hat Jira (volcado Zenodo de aproximadamente 266 MB tras descomprimir), GHArchive (eventos de GitHub a escala de terabytes mediante streaming) y un dataset sintético a generar por el propio equipo.

Decisión. El lakehouse se construye exclusivamente sobre PROMISE y Red Hat Jira.

Alternativas evaluadas. Apache Jira en MongoDB fue descartado porque su volumen y el formato requieren un pipeline de descompresión y normalización (BSON a Parquet) cuyo costo no aporta valor analítico incremental respecto a Red Hat, que ya provee la misma vista del proceso de Jira. GHArchive fue descartado porque su escala impone un patrón de streaming incompatible con el alcance batch del trabajo y porque su ingesta requeriría infraestructura adicional fuera del alcance de Databricks Free Edition. El dataset sintético fue descartado porque no existía al momento de iniciar el proyecto y porque su generación implica un sub-proyecto autónomo (motor de simulación calibrado con SimPy) que no estaba alineado con los entregables académicos de este trabajo.

Consecuencia. La combinación PROMISE más Red Hat enriquece la arquitectura con dos dominios complementarios (código y proceso) que habilitan la construcción de `dim_project`. Esta decisión convierte el lakehouse de un caso académico monocliente a una arquitectura *multi-source* con joins cross-domain genuinos.

DD-002 Exclusión del archivo RHD.csv

Contexto. El dataset Red Hat contiene 251 archivos CSV, uno por sistema. El *schema fingerprinting* sobre los 251 archivos reveló que 250 comparten un esquema canónico de nueve columnas (Issue key, Issue Type, Status, Project key, Project name, Project type, Resolution, Created, Resolved) mientras que RHD.csv expone un esquema divergente con Assignee, Updated y Fix version/s en lugar de Resolution, Created y Resolved.

Decisión. El archivo RHD.csv se excluye de la ingesta y se preserva localmente en `data/raw/redhat_excluded_RHD.csv` como evidencia de trazabilidad.

Alternativas evaluadas. La unión con *NULL padding* sobre las columnas no compartidas habría contaminado Silver con métricas faltantes y habría impedido aplicar la constraint `cycle_time_non_negative` ya que el cálculo del cycle time depende de columnas que RHD.csv no contiene. La creación de una segunda tabla Bronze (`redhat_pbi_special_raw`) específicamente para este único archivo fue evaluada y descartada por sobreingeniería: dedicar una tabla a un único archivo de 24 KB con esquema único contradice los principios de parsimonia del modelo dimensional.

Consecuencia. El lakehouse pierde visibilidad sobre el sistema RHD (*Red Hat Developer*), que representa el 0.4 % de los archivos disponibles. Las columnas no compartidas (`Assignee`, `Fix version/s`) no son críticas para el análisis dado que las restricciones éticas del PI prohíben análisis a nivel de desarrollador individual.

DD-003 Filtrado de filas con resolved < created

Contexto. El chequeo pre-DQ sobre Bronze de Red Hat detectó 125 issues con `resolved` estrictamente anterior a `created`, una condición físicamente imposible que sugiere errores de captura, retroactividad manual o migración de tickets entre sistemas. Dejar pasar estas filas habría producido valores negativos en la columna derivada `cycle_time_days`, contaminando las agregaciones temporales del EDA.

Decisión. En la transformación Bronze→Silver se aplica el filtro (`resolved IS NULL OR resolved >= created`), y la cantidad exacta de filas eliminadas se registra como evento `rows_dropped_inverted_dates` en `_dq_metrics` para trazabilidad.

Alternativas evaluadas. Conservar las filas con `cycle_time_days` forzado a NULL fue evaluado: el coste es contaminar la columna `is_resolved` con casos donde el issue figura como resuelto pero su métrica de cycle no es interpretable. Eliminar la constraint `cycle_time_non_negative` fue evaluado y descartado: significaría renunciar a la garantía física sobre Silver, contradiciendo el principio de la capa como *layer of guaranteed invariants*.

Consecuencia. Se descarta el 0.024 % del dataset Red Hat a cambio de constraints físicas que garantizan invariantes matemáticas en las agregaciones temporales.

DD-004 Pipeline ML híbrido por restricciones de Free Edition

Contexto. Databricks Free Edition Serverless impone, de forma acumulativa, cuatro restricciones de runtime que afectan la implementación de un pipeline de ML clásico sobre SparkMLlib distribuido.

La primera es una *whitelist* de Py4J que bloquea la construcción directa de los *constructors* JVM de las clases tradicionales `pyspark.ml.feature.VectorAssembler`,

`pyspark.ml.feature.StandardScaler`, `pyspark.ml.classification.LogisticRegression`, `pyspark.ml.classification.RandomForestClassifier`, `pyspark.ml.classification.GBTClassifier` y `pyspark.ml.evaluation.BinaryClassificationEvaluator`. La excepción es lanzada con el mensaje `Py4JSecurityException: Constructor ... is not whitelisted`.

La segunda restricción es la prohibición de `.cache()` y `.persist()` sobre `DataFrames`, que arrojan `NOT_SUPPORTED_WITH_SERVERLESS`. Free Edition delega el manejo de caché a Photon y al optimizador predictivo, sin permitir control manual.

La tercera restricción es que la API moderna `pyspark.ml.connect.LogisticRegression`, único clasificador disponible en la whitelist, se implementa internamente como un *wrapper* sobre PyTorch. PyTorch no viene preinstalado en el runtime y debe instalarse en una primera celda mediante `%pip install torch`.

La cuarta restricción se manifiesta tras instalar PyTorch: `pyspark.ml.connect.LogisticRegression.fit()` intenta leer la configuración `spark.master`, que está deliberadamente oculta en Serverless por razones de seguridad, y arroja `CONFIG_NOT_AVAILABLE: Configuration spark.master is not available`.

Decisión. El pipeline ML adopta una arquitectura híbrida. La ingeniería de features (z-score scaling y construcción del vector de features mediante `F.array()`) se mantiene íntegramente distribuida en SparkSQL sobre las tablas Silver. Los modelos se entrenan en el driver con `scikit-learn` (`RandomForestClassifier` y `GradientBoostingClassifier`) tras materializar el conjunto de entrenamiento con `.toPandas()`; el dataset, con 15,775 filas y veinte features, ocupa aproximadamente 3 MB en memoria, una magnitud trivial para el driver. La búsqueda de hiperparámetros se realiza con `GridSearchCV` y validación cruzada estratificada de cinco folds. El modelo campeón se persiste en el Volume con `joblib` y un `metadata.json` adjunto.

Alternativas evaluadas. Migrar a Databricks Premium con ML Runtime habría desbloqueado la API clásica completa, pero está fuera del alcance del enunciado (“Databricks Community/Free Edition”). Usar XGBoost en lugar de `sklearn` fue evaluado pero presenta exactamente el mismo problema de runtime al requerir Spark distribuido. Una migración a Spark Connect standalone fuera de Databricks (instalando Spark 4.0+ con PyTorch en un entorno local) habría desbloqueado la API moderna, pero contradice el requisito de plataforma del enunciado.

Consecuencia. El feature engineering distribuido y la lectura escalable de datos sobre Spark se preservan; únicamente el solver del modelo opera en el driver. Para un dataset del tamaño actual esta diferencia es operativamente irrelevante. Para datasets mayores se recomienda migrar a Databricks Premium (no Free) o adoptar `applyInPandas` sobre Spark para particionar el entrenamiento.

Mitigaciones futuras. El repositorio queda preparado para una migración trivial al runtime Premium: las variables de entorno y la estructura de scripts son idénticas, y únicamente requeriría reemplazar el bloque de entrenamiento `sklearn` por su equivalente `pyspark.ml.classification.RandomForestClassifier`, manteniendo el resto del pipeline intacto.

11. Reproducibilidad

11.1. Estructura del repositorio

El repositorio está organizado siguiendo convenciones estándar de proyectos enterprise de ingeniería de datos. La carpeta `src/` contiene los pipelines como notebooks Python ejecutables en Databricks y los scripts SQL ejecutables vía Statements API. La carpeta `scripts/` contiene los *helpers* de descarga, subida al Volume e inspección operacional, todos ellos en bash o Python ejecutable. La

carpeta `docs/` contiene la bitácora formal de decisiones (`data_decisions.md` con DD-001 a DD-004), la presente memoria LaTeX y, una vez generado, el PDF compilado. La carpeta `notebooks/exports/` preserva los HTMLs ejecutados de Databricks como evidencia inmutable de cada corrida exitosa. Los datasets crudos en `data/raw/` están excluidos del control de versiones mediante `.gitignore` y se reconstruyen ejecutando los scripts de descarga.

11.2. Pasos para reproducir el lakehouse desde cero

La reproducción completa desde un clon limpio del repositorio requiere únicamente cinco pasos. El primero consiste en establecer los prerequisites locales: Git versión 2.30 o superior, Python 3.10 o superior y la Databricks CLI versión 0.240 o superior. El segundo consiste en crear una cuenta en Databricks Free Edition y generar un *Personal Access Token* desde la sección *User Settings*. El tercero exporta las variables de entorno requeridas por los scripts: `DATABRICKS_HOST`, `DATABRICKS_TOKEN`, `WAREHOUSE_ID` y `DBX_USER`. El cuarto ejecuta el script de bootstrap del lakehouse, los scripts de descarga y el script de subida al Volume, en ese orden. El quinto sube cada notebook de `src/pipelines/` al workspace mediante `databricks workspace import` y los ejecuta en orden numérico desde la interfaz de Databricks.

11.3. Idempotencia

Todos los pipelines fueron diseñados para ser idempotentes desde el inicio. Los schemas y el Volume se crean con `IF NOT EXISTS`. Las tablas Delta se escriben con `mode(“overwrite”) y overwriteSchema=true, lo cual permite re-ejecutar cualquier notebook sin riesgo de corrupción. Las constraints se aplican mediante un helper que captura la excepción already exists y la convierte en un mensaje informativo no bloqueante. La tabla _dq_metrics opera en modo append con un run_id único por ejecución, permitiendo conservar el histórico de todas las corridas y comparar runs entre sí.`

12. Limitaciones y trabajo futuro

Las cuatro restricciones de runtime documentadas en DD-004 son la limitación principal del trabajo y la razón por la cual el modelo no se entrena con el solver distribuido nativo de SparkMLlib. Para datasets de escala superior a 100,000 filas en producción se recomienda migrar a Databricks Premium con ML Runtime, lo cual desbloquea la API clásica completa, o alternativamente adoptar el patrón `applyInPandas` para particionar el entrenamiento por shards mantenidos sobre Spark.

A nivel de modelo, las limitaciones identificadas y su resolución sugerida son tres. El recall bajo sobre la clase positiva sugiere que para uso productivo en CI/CD sería necesario aplicar re-balanceo de clases (SMOTE o ajuste de `class_weight`) o tuning del threshold; en el contexto de SLT, donde el costo de un falso negativo es significativamente mayor que el de un falso positivo, esto es una mejora de primer orden. La validación temporal es la segunda limitación: el split actual es aleatorio, lo cual no captura la naturaleza secuencial de las versiones de software; una mejora directa consistiría en validar con `TimeSeriesSplit` entrenando sobre versiones tempranas y prediciendo sobre versiones posteriores. La tercera limitación es que el modelo utiliza únicamente features estáticas de código; integrar las features de proceso del dominio Red Hat (cycle time, throughput, tipo de issue) cruzadas vía `dim_project` permitiría un modelo verdaderamente cross-domain alineado con la narrativa original del Proyecto Integrador ShiftMetrics.

Como líneas de trabajo futuro a más largo plazo, el repositorio queda preparado para integrar *MLflow* como registro centralizado de modelos, lo cual añadiría *model versioning* y *model registry* sobre el actual sistema de artefactos en Volume; para integrar SHAP values en el dashboard de aplicación, permitiendo al consumidor entender por qué cada módulo se clasifica como riesgoso; y para migrar la orquestación de los pipelines a Databricks Workflows o Asset Bundles, lo cual añade ejecución agendada, alertas operacionales y *dependency tracking* entre etapas.

13. Conclusiones

ShiftMetrics Lakehouse demuestra la factibilidad de construir una arquitectura batch enterprise-grade sobre Databricks Free Edition combinando el patrón Medallion canónico con adaptaciones explícitamente documentadas a las restricciones del runtime gratuito. La solución final procesa 520,415 registros provenientes de dos dominios complementarios del ciclo de vida del software, materializa diecinueve tablas Delta organizadas en tres capas, aplica diecinueve chequeos de Data Quality por corrida con cero fallas críticas, declara seis constraints físicas que actúan como invariantes matemáticas sobre los datos curados, entrena tres candidatos de Machine Learning bajo validación cruzada con búsqueda en grid de hiperparámetros, persiste el modelo campeón con su escalador y metadata en un Volume gestionado por Unity Catalog, y entrega como output accionable una priorización de once proyectos open-source para la adopción de Shift-Left Testing.

La separación estricta entre capas mediante schemas independientes y la materialización de cada KPI como tabla Delta con *comment* descriptivo permite que cualquier consumidor del lakehouse navegue su contenido desde el Catalog Explorer sin requerir documentación externa. La bitácora formal de decisiones de arquitectura DD-001 a DD-004 proporciona la justificación auditable de cada compromiso técnico no trivial, convirtiendo las limitaciones del runtime en evidencia documentada en lugar de obstáculos opacos. El repositorio resultante es reproducible de extremo a extremo en cinco pasos y queda preparado para evolucionar hacia un entorno Databricks Premium sin reescritura sustancial.

Más allá del cumplimiento del enunciado académico, el trabajo entrega una lección operativa: en plataformas restrictivas como Databricks Free Edition, la calidad de una arquitectura de datos se mide no por la ausencia de obstáculos sino por la transparencia con la que se documentan y se mitigan. Las cuatro restricciones acumuladas del runtime no comprometieron el patrón Medallion; únicamente desplazaron el motor de entrenamiento del cluster al driver, una decisión cuyas implicaciones son explícitas y reversibles.